

Hermes Agent Sub-Agent 体系设计

1. 体系总览
2. 核心架构: Delegate Task
 - 概述
 - 关键设计常量
 - 两种运行模式
3. 生命周期与隔离机制
 - 子 Agent 的构建
 - 上下文隔离策略
 - 工具权限限收紧
4. 中断传播机制
 - 三层中断架构
 - 中断传播流程
 - 中断传播的线程安全性
5. 凭证与 Provider 解析
 - 子 Agent 的凭证继承
 - Provider 解析优先级
6. 进度反馈与 UI 集成
 - 双路经进度回调
 - `_last_resolved_tool_names` 全局污染问题
7. 迭代预算
 - 预算隔离设计
 - `execute_code` 退款机制
8. Tool Call 截断保护
9. RL 环境 Agent Loop
 - 差异对比
 - 工具调度差异
10. ACP 会话管理器
11. Mixture-of-Agents
12. 设计理念总结
 - 核心原则
 - 与传统 Multi-Agent 的区别
13. 注意事项与已知陷阱
 - 开发注意事项
 - 安全注意事项
 - 性能注意事项
14. 文件索引

Hermes Agent 的 Sub-Agent 体系并非一个"Agent框架"式的多智能体系统，而是一套基于工具调用的任务委托（Delegation）机制。其核心由以下几个模块组成：

模块	文件路径	职责
delegate_task 工具	tools/delegate_tool.py	Sub-agent 的发起、生命周期管理、并发控制
AIAgent 核心类	run_agent.py	Sub-agent 的执行引擎、对话循环、中断传播
中断信号机制	tools/interrupt.py	跨线程中断通知（全局 threading.Event）
辅助客户端路由	agent/auxiliary_client.py	Sub-agent 的 Provider/Model 解析与凭证管理
环境 Agent 循环	environments/agent_loop.py	RL 训练环境的专用 agent 循环
ACP 会话管理器	acp_adapter/session.py	VS Code 等编辑器的多会话管理
Mixture-of-Agents	tools/mixture_of_agents_tool.py	参考多模型的聚合推理工具（独立于 delegation）

2.1 概述

delegate task 是作为一个**工具**注册的 AI Agent 子系统（属于 delegation toolset）。主 Agent 通过调用此工具来生成子进程级别的 sub-agent，每个子 agent 拥有：

- 独立的对话历史（隔离上下文）
- 独立的终端会话（独立 `task_id`）
- 受限的工具集（隔离权限）
- 专注的系统提示（Focused System Prompt）

子 agent 的中间工具调用不会泄漏到父 agent 的上下文，仅返回最终摘要（即上下文隔离）

2.2 关键设计常量

```
# tools/delegate_tool.py
DELEGATE_BLOCKED_TOOLS = frozenset([
    "delegate_task",      # 禁止递归委托
    "clarify",            # 禁止用户交互
    "memory",             # 禁止写入共享 MEMORY.md
    "send_message",       # 禁止跨平台副作用
    "execute_code",       # 子 agent 应逐步推理，而非写脚本
])
```

```
MAX_CONCURRENT_CHILDREN = 3    # 最大并发子 agent 数量
```

```
MAX_DEPTH = 2                # 委托深度限制（父=0，子=1，孙=2 被拒绝）
DEFAULT_MAX_ITERATIONS = 50   # 每个子 agent 的默认最大迭代次数
DEFAULT_TOOLSETS = ["terminal", "file", "web"] # 默认工具集
```



Note

MAX_CONCURRENT_CHILDREN=3：认知边界，而非资源上限

3 这个数字的来源不是技术约束，而是产品直觉：

- 超过 3 个并行动作，用户无法有效追踪系统行为
- 这与“7±2 法则”同源——人对并行流的认知容量极为有限
- 超过 3 的并发拆分需求，本质上是 workflow 设计问题，不应通过叠加委托深度解决

Note

MAX_DEPTH=2：信任链，而非调用栈

深度限制的语义不是技术栈深度，而是**可见性边界**：

- 子 agent 已经引入了信息损耗（摘要替代细节），再委托一层后，父对孙的摘要无法判断质量
- 委托链的错误传播路径跨两层后变得不可追踪
- MAX_DEPTH=2 的本质约束：只有当父能看到子的完整工作时，委托才有意义

Note

截断算法的设计原则：并发约束只针对委托行为本身

- 并发限制约束的是“创建子 agent”的行为，不是所有工具调用
- 同一响应中的非 delegate 工具（terminal、web_search 等）不受截断影响
- 截断与去重构成双层防护：先截断 delegate，再对结果去重，防止重复调用

2.3 两种运行模式

单任务模式（Single Task）：

```
delegate_task(goal="修复 bug #123", context="...", toolsets=["terminal", "file"])
```



直接在主线程执行，无线程池开销。

批处理模式（Batch, Parallel）：

```
delegate_task(tasks=[
    {"goal": "任务A", "context": "..."},
    {"goal": "任务B", "context": "..."},
    {"goal": "任务C", "context": "..."},
])
```



通过 ThreadPoolExecutor 并发执行，最多 MAX_CONCURRENT_CHILDREN=3 个并行。

Note

Provider 链：能力路由，而非兜底备援

不同任务类型的路由逻辑不同，因为瓶颈不同：

- 文本任务：瓶颈是“模型选择空间” → OpenRouter 最优（数百个模型）
- 视觉任务：瓶颈是“多模态支持完整性” → 主 Provider 最优（PDF、图像、交错输入）
- 视觉任务不经过 OpenRouter 聚合层：第三方视觉路由不可靠，主 Provider 的原生支持更完整

Note

批处理结果：用户感知有序，内部并发无阻塞

- 并发执行，但返回顺序与输入顺序一致（确定性）
- 核心算法：“先到先解包，后按原序返回”——用 task_index 映射解耦完成顺序与返回顺序
- 单任务时跳过线程池，直接主线程执行，消除不必要的开销

三、生命周期与隔离机制

3.1 子 Agent 的构建（_build_child_agent）

子 agent 由父 agent 在**主线程**同步构建（线程安全），再交由线程池运行：

```
# tools/delegate_tool.py:150-249
child = AIAgent(
    base_url=effective_base_url,
    api_key=effective_api_key,
    model=effective_model,
    provider=effective_provider,
    max_iterations=max_iterations,      # 独立迭代预算
    enabled_toolsets=child_toolsets,    # 受限工具集（与父交集）
    quiet_mode=True,                   # 静默模式
    ephemeral_system_prompt=child_prompt, # 专注型系统提示
    log_prefix=f"[subagent-{task_index}]",
    skip_context_files=True,           # 跳过上下文文件
    skip_memory=True,                  # 跳过记忆系统
    session_db=parent_agent.session_db, # 共享会话数据库
    tool_progress_callback=child_progress_cb, # 进度回调
    iteration_budget=None,              # 每个子 agent 有独立预算
)
```

```
child._delegate_depth = parent._delegate_depth + 1
```



3.2 上下文隔离策略

隔离的内容：

- 父 agent 的对话历史（完全不继承）
- 独立生成的系统提示（Focused prompt）
- 独立的 task_id（独立终端会话状态）
- 独立的迭代预算（IterationBudget）

继承的内容：

- API 凭证（base_url、api_key 等）
- 父 agent 的 enabled_toolsets **交集**（不能超出父的能力）
- 父 agent 的 providers 配置

阻断的内容（强制移除）：

- delegate_task（防止递归）
- clarify、memory、send_message、execute_code

3.3 工具集权限收紧（Intersection 策略）

```
# tools/delegate_tool.py:177-184
parent_toolsets = set(parent_agent.enabled_toolsets or DEFAULT_TOOLSETS)
if toolsets:
    child_toolsets = _strip_blocked_tools([t for t in toolsets if t in parent_toolsets])
elif parent_agent.enabled_toolsets:
    child_toolsets = _strip_blocked_tools(parent_agent.enabled_toolsets)
else:
    child_toolsets = _strip_blocked_tools(DEFAULT_TOOLSETS)
```



Sub-agent 只能获得父 agent 已有且未被 blocked 的工具。**不允许扩大权限**，体现了最小权限原则。

Note

工具集层过滤 vs. 工具名层过滤：命名约定安全与精确封锁的纵深防御

有两套独立的过滤机制：

- 工具集层：按工具集名称过滤整个权限集合（如 delegation、clarify）
- 工具名层：按具体工具名精确封锁（如 delegate_task、clarify）

命名约定过滤更简洁，但脆弱（工具集重名则封锁失效）；精确封锁更健壮。两者组合形成纵深防御。

Note

Intersection 空集：静默降级，而非快速失败

当用户请求的工具集与父 agent 的交集为空时：

- 不会报错，而是让子 agent 以空工具集运行
- 子 agent 执行时隐性失败，返回 "failed" 状态
- 这是"宁可少给工具，不能越权给工具"的安全默认值
- 代价：错误不在委托时暴露，需要深入子 agent 日志才能发现配置错误

四、中断传播机制（Interrupt Propagation）

4.1 三层中断架构

Hermes Agent 实现了一套三层中断传播机制，确保当用户终止父 agent 时，所有子 agent 能立即感知并退出。

三层各有明确的职责分工：Layer 1 负责跨线程通知（任意工具在执行中都能感知中断），Layer 2 负责控制 agent 主循环的迭代决策，Layer 3 负责将中断信号递归传播到所有子 agent。三者协同，保证中断信号既能在细粒度的工具执行层被响应，又能在粗粒度的 agent 决策层被处理。

Layer 1 — 全局中断事件（tools/interrupt.py）：

进程级共享，所有工具在执行中都可查询。用于耗时操作（代码执行、外部进程）提前退出。

```
_interrupt_event = threading.Event() # 全局单例
```

```
def set_interrupt(active: bool): ...
def is_interrupted() -> bool: ...
```



Layer 2 — AI Agent 实例中断标志（run_agent.py）：

实例级标志，由 interrupt() 方法和主循环边界读写。用于决定是否继续下一轮迭代。

```
class AI Agent:
    self._interrupt_requested = False
    self._interrupt_message = None
```



Layer 3 — 子 Agent 链表（run_agent.py）：

将中断信号从父 agent 递归下发到每个子 agent 的实例集合。线程安全。

```
self._active_children = [] # 运行中的子 AI Agent 列表
self._active_children_lock = threading.Lock()
```



4.2 中断传播流程

用户输入新消息 或 Ctrl+C

↓

父 agent.interrupt(message)

↓

1. 设置 self._interrupt_requested = True
2. 设置全局 _interrupt_event (通知所有工具)
3. 遍历 _active_children, 对每个子 agent 递归调用 interrupt()

↓

子 agent 收到 interrupt()

↓

子 agent.run_conversation() 中的各检查点检测到 _interrupt_requested:

- _interruptible_api_call(): 在 HTTP 线程 join 循环中检查
- _interruptible_streaming_api_call(): 在流式循环中检查
- 抛出 InterruptedError, 停止当前迭代

↓

_run_single_child() 捕获异常, 返回 status="interrupted"



Note

中断语义：意图切换，而非操作中止

用户触发中断时，心理预期是“立即停止”，而非“等待完成”：

- 部分积累的 token 可以丢弃（新的意图不需要未完成的输出）
- 完整积累的工具调用参数必须保留（这是后续执行的安全输入）
- 保真度优先级：工具调用参数 > 流式 token
- 本质上是将“中断”理解为意图切换，正确选择了保留哪些状态

Note

三层中断：感知总线与决策信号的分层授权

- Layer 1 (_interrupt_event)：工具可见层，被动感知——任何工具可在执行中主动查询并提前退出
- Layer 2 (_interrupt_requested)：agent 实例层，主动决策——在循环边界（API 调用前后）检查，决定是否继续迭代
- Layer 3 (_active_children 链表)：传播层——递归通知所有子 agent
- 两层信号组合：既能在工具执行期间被感知，又能在 agent 决策点被响应

4.3 中断传播的线程安全性

- _active_children_lock 是 threading.Lock，保护子 agent 链表的并发访问
- 子 agent 在 _run_single_child 的 finally 块中将自己从 _active_children 移除
- interrupted 状态会正确反映在委托工具的 JSON 返回值中

Note

流式中断延迟：最多一个 chunk，而非一个请求

- 插入流式循环检查中断，而非使用阻塞的线程 join——不引入额外等待
- 响应延迟上限：min(下一个 chunk 到达时间, STREAM_READ_TIMEOUT)
- 这是中断及时性 with 流式吞吐之间的务实权衡

五、凭证与 Provider 解析

5.1 子 Agent 的凭证继承

delegate_task 支持为子 agent 配置**独立的 Provider/Model**，而非简单继承父 agent 的设置：

```
# tools/delegate_tool.py:570-656
def _resolve_delegation_credentials(cfg, parent_agent) -> dict:
    # 三种配置方式：
    # 1. delegation.base_url → 直接 OpenAI-compatible 端点
    # 2. delegation.provider → 通过 runtime_provider 系统解析
    # 3. 无配置 → 完全继承父 agent 的凭证
```



支持通过 delegation.provider 将子 agent 路由到不同的 Provider:Model 组合，例如：

- 父 agent 运行在 Nous Portal（强推理模型）
- 子 agent 可以路由到 OpenRouter（快速廉价模型）

5.2 Provider 解析优先级

agent/auxiliary_client.py 中定义了完整的辅助任务 Provider 链：

文本任务自动检测顺序：

1. OpenRouter (OPENROUTER_API_KEY)
2. Nous Portal (OAuth, ~/.hermes/auth.json)
3. 自定义端点 (config.yaml + OPENAI_API_KEY)
4. Codex OAuth (Responses API)
5. Native Anthropic
6. 直接 API Key providers (z.ai/GLM、Kimi、MiniMax)

视觉任务顺序：

1. 主 Provider（如果是已知视觉后端）
2. OpenRouter → Nous → Codex → Anthropic → Custom

六、进度反馈与 UI 集成

6.1 双路径进度回调

```
# tools/delegate_tool.py:78-147
def _build_child_progress_callback(task_index, parent_agent, task_count) -> callable:
    # 返回一个 callback(tool_name, preview=None)

    # 路径 1: CLI (KawaiiSpinner 树形视图)
    if spinner:
        spinner.print_above(f" {prefix}|─ 🌀 {tool_name}: {preview}")

    # 路径 2: Gateway (批量工具名聚合)
    if parent_cb:
        _batch.append(tool_name)
        if len(_batch) >= 5: # 批量大小
            parent_cb("subagent_progress", f"🌀 {summary}")
        # 完成后 flush 剩余
```



- CLI 模式：通过 KawaiiSpinner.print_above() 在 spinner 上方打印树形进度
- Gateway 模式：通过 tool_progress_callback 批量中转给消息平台
- 工具回调 (tool_progress_callback) 通过 batch 聚合避免消息泛滥

Note

子 agent 不继承对话历史：交接文档，而非延续记忆

核心假设：子任务不需要父任务的过程记忆，只需要目标和背景约束

- 类比人类工作：交接文档不记录过程，只记录结论和约束
- 委托时强制父 agent 进行“认知压缩”——写不出简洁描述意味着任务边界本身不清晰
- 副作用：长程推理任务（需要延续探索路径）可能因隔离而产生重复探索

Note

双路径进度反馈：不同平台，不同注意力模型

- CLI 用户：开发者，专注等待，终端空间充裕，需要推理过程可见性（_thinking 摘要可见，树形缩进合理）
- Gateway 用户：异步通知，注意力碎片化，不需要推理日志，只需要知道“正在工作”
- batch_size=5: 实时性与消息通道压力的权衡（单条发送最好，但部分平台有频率限制）

Note

_thinking 事件：CLI 可见，Gateway 有意过滤

- _thinking 内容在 CLI 显示前 55 字符，帮助用户理解推理过程
- Gateway（Slack/飞书等）不 relay _thinking：推理日志形成噪音消息，受众不需要流式推理细节
- _flush() 机制：仅在 Gateway 路径调用，确保子 agent 退出时最后不足 batch_size 的事件也被发送

6.2 _last_resolved_tool_names 全局污染问题

问题描述：
model_tools._last_resolved_tool_names 是一个**进程级全局变量**，在 _build_child_agent() 调用 AIAgent.__init__() → get_tool_definitions() 时会被子 agent 的工具集覆盖，导致父 agent 后续的 execute_code 等工具拿到错误的工具列表。

解决方案（delegate_tool.py 中实现）：

```
# tools/delegate_tool.py:471-496
# 在构建任何子 agent 之前保存父工具名
_parent_tool_names = list(model_tools._last_resolved_tool_names)

try:
    for i, t in enumerate(task_list):
        child = _build_child_agent(...)
        child._delegate_saved_tool_names = _parent_tool_names # 挂载到 child 对象
        children.append((i, t, child))
finally:
    # 即使子 agent 构建失败，也要恢复全局状态
    model_tools._last_resolved_tool_names = _parent_tool_names

# 在 _run_single_child 的 finally 块中恢复
finally:
    saved_tool_names = getattr(child, "_delegate_saved_tool_names", None)
    if isinstance(saved_tool_names, list):
        model_tools._last_resolved_tool_names = list(saved_tool_names)
```



七、迭代预算（IterationBudget）

7.1 预算隔离设计

```
# run_agent.py:170-180
class IterationBudget:
    """线程安全的迭代计数器。

    父 agent 的预算上限为 max_iterations (默认 90)。
    每个子 agent 有独立的预算上限 delegation.max_iterations (默认 50)。
    父 + 子 agent 的总迭代次数可以超过父的 max_iterations。
    """
```



关键设计决策：

- 子 agent **不共享**父的迭代预算（iteration_budget=None → 新建独立预算）
- 这样设计是为了防止子 agent 消耗完父的迭代预算，导致父无法完成收尾工作
- 用户可通过 delegation.max_iterations 配置子 agent 预算上限

Note

迭代预算隔离：能力边界设计，而非账单保护

- 预算独立 = 最大能力释放，而非成本保护（总迭代次数可以远超父的 max_iterations）
- 共享预算的风险：子 agent 可能耗尽父的预算，导致父无法完成收尾工作
- execute_code 退款逻辑：纯脚本编写轮次不消耗推理预算——token 留给真正需要 LLM 推理的步骤

7.2 execute_code 退款机制

```
# run_agent.py:7746-7748
_tc_names = {tc.function.name for tc in assistant_message.tool_calls}
if _tc_names == {"execute_code"}:
    self.iteration_budget.refund() # 退款，不计入迭代限制
```



当一个迭代轮次只调用了 execute_code（程序化工具调用），该迭代不消耗预算，防止因编写与执行脚本而耗尽迭代次数。

Note

execute_code 退款：精确相等，而非子集检查

- 精确集合相等：只有本轮唯一工具调用是 execute_code 才退款
- execute_code + 任何其他工具 → 不退款（视为“用脚本辅助操作”，正常计费）
- 这是一个白盒信任设计：纯写脚本本轮次不需要消耗推理预算

八、Tool Call 截断保护

```
# run_agent.py:2807-2835
@staticmethod
def _cap_delegate_task_calls(tool_calls: list) -> list:
    """将多余的 delegate_task 调用截断到 MAX_CONCURRENT_CHILDREN。

    防止模型在一次响应中多次调用 delegate_task，
    绕过 MAX_CONCURRENT_CHILDREN=3 的并发限制。
    """
```



即使模型在单次 LLM 响应中生成了多个 delegate_task 调用，系统也会自动截断到最多 3 个，防止资源耗尽攻击。

environments/agent_loop.py 中的 HermesAgentLoop 是用于强化学习训练环境的专用 agent 循环，与主 agent(run_agent.py) 共享工具调度逻辑，但有独立设计：

9.1 差异对比

特性	run_agent.py AI Agent	HermesAgentLoop
执行模式	同步（主线程阻塞式）	异步（async def run()）
工具执行	同步 handle_function_call()	loop.run_in_executor() 线程池
线程池大小	主线程	可配置（resize_tool_pool(), 默认 128）
状态持久化	SessionDB	ManagedServer（RL 环境）
中断机制	线程中断标志	无（RL 环境不适用）
TodoStore	全局	每轮新建 Ephemeral

9.2 工具调度差异

```
# environments/agent_loop.py:394-407
# Todo/memory/session_search 由 HermesAgentLoop 内部特殊处理
if tool_name == "todo":
    tool_result = _todo_tool(todos=args.get("todos"), store=_todo_store)
elif tool_name == "memory":
    tool_result = '{"error": "Memory is not available in RL environments."}'
else:
    loop = asyncio.get_event_loop()
    tool_result = await loop.run_in_executor(
        _tool_executor, # 128 工作线程的线程池
        lambda: handle_function_call(tool_name, args, task_id=task_id),
    )
```



十、ACP 会话管理器（多编辑器集成）

acp_adapter/session.py 的 SessionManager 提供了 IDE 集成场景下的多会话管理：

- 每个 VS Code / Zed / JetBrains 会话对应一个独立的 SessionState
- 每个会话包含一个 AI Agent 实例
- 支持会话的 fork（深拷贝历史到新会话）
- 支持会话的持久化（重启后恢复）
- 每个会话有独立的 cancel_event（线程级取消）

十一、Mixture-of-Agents（独立于 Delegation）

tools/mixture_of_agents_tool.py 实现了基于研究论文 [arXiv:2406.04692v1](#) 的 MoA 架构，与 delegate_task 完全独立：

两层架构：

Layer 1（参考模型，并行）：
- claude-opus-4.6 / gemini-3-pro / gpt-5.4-pro / deepseek-v3.2

Layer 2（聚合模型）：
- claude-opus-4.6 → 合成最终答案



- 每个参考模型通过 OpenRouter API 并行调用

- 失败容错：MIN_SUCCESSFUL_REFERENCES=1（至少 1 个成功即可）
- 重试机制：指数退避（最多 6 次）
- 仅适用于极度复杂、需要多角度推理的任务

Note

MoA vs. Delegation：提升质量 vs. 扩大能力

关键区别在于**信息流方向**：

- Delegation：发散-汇聚（父→多子→汇总），适合“做更多事”
- MoA：并行（多独立源→单一聚合器），适合“把一件事做得更正确”

两者互补——未来方向：先用 MoA 为子任务生成候选方案，再用 Delegation 分发给专业 agent 执行

Note

MoA 的关键参数设计

- 宽松闸门（MIN_SUCCESSFUL_REFERENCES=1）：≥1 个参考模型成功即可继续，允许 4 个中失败 3 个
- 容错内化：每个模型在内部捕获异常返回 False，gather 不会因单模型失败而整体崩溃
- 独立指数退避：各模型重试独立进行，最坏等待时间 = max(各模型耗时)，而非 sum
- 温度不对称：参考层 0.6（高多样）→ 聚合层 0.4（低温度忠实整合）

Note

体系主线：受控开放（Controlled Openness）

三个信任止点，构成渐进式权力下放：

- 父对子：信任止于摘要（不信任中间过程），子不能委托
- 子对自己：信任止于迭代预算（不信任无限执行），不能 clarify
- 系统对模型：信任止于工具截断（不信任异常响应）

这与行业“赋予 agent 越多能力越好”的主流趋势相悖——Hermes Agent 的选择是**能力越强，约束越紧**，在 AI Safety 视角下更具防御性。

十二、设计理念总结

12.1 核心原则

1. **最小权限隔离（Least Privilege）**：Sub-agent 不能获得父 agent 缺少的工具；Blocked tools 强制移除
2. **上下文净化（Context Purity）**：子 agent 的中间结果不泄漏到父上下文，防止上下文污染
3. **资源可控（Resource Bounded）**：深度限制（MAX_DEPTH=2）、并发限制（MAX_CONCURRENT=3）、迭代限制
4. **线程安全（Thread Safety）**：通过锁、全局状态保存/恢复、interrupt 事件实现跨线程协作
5. **凭证灵活路由（Flexible Routing）**：子 agent 可独立于父选择不同的 Provider/Model
6. **渐进式降级（Graceful Degradation）**：Provider 自动回退、批处理中断时部分结果返回

12.2 与传统 Multi-Agent 的区别

维度	传统 Multi-Agent 系统	Hermes Sub-Agent
通信协议	Agent 间直接消息传递	通过工具调用（tool_call）
状态共享	共享记忆/黑板	完全隔离，独立预算
层级关系	扁平或层级化	树形层级（MAX_DEPTH=2）
上下文	相互可读	父只能看到摘要
适用场景	开放协作、辩论	任务分解、独立执行

十三、注意事项与已知陷阱

13.1 开发注意事项

1. **_last_resolved_tool_names 全局污染**
 - 在 delegate_tool.py 中已通过保存/恢复机制解决，但任何新增的进程级全局变量都可能引发类似问题
 - 新增代码如依赖此全局，必须考虑子 agent 构建时的覆盖风险
2. **深度限制的隐式行为**
 - 父 agent 的 _delegate_depth=0，子=1，孙=2 被拒绝
 - 当父 agent 的任务触发两次 delegate_task（第一次委托后子再委托），第二次会静默失败
3. **批处理返回顺序**
 - 结果按 task_index 排序返回，确保与输入顺序一致（results.sort(key=lambda r: r["task_index"]））
4. **Sub-agent 的 iteration_budget 独立性**
 - 父 agent 的 90 次迭代和子 agent 的 50 次迭代**独立计算**
 - 总迭代次数可以远超 90，用户需注意成本控制
5. **Gateway 进度回调的 batch 延迟**
 - Gateway 进度回调每 5 个工具名才发送一次
 - 最后剩余不足 5 个的通过 _flush() 在子 agent 完成后发送
6. **execute_code 预算退款**
 - 仅当本轮**唯一**工具调用是 execute_code 时才退款
 - 如果 execute_code 和其他工具混在一轮，不退款
7. **ACP 会话的 CWD 覆盖**
 - ACP 适配器通过 register_task_env_overrides() 为每个 task_id 单独注册 CWD
 - 不同编辑器会话共享同一个 ~/.hermes/state.db，通过 session_id 隔离

13.2 安全注意事项

- 1. 禁止在 Sub-agent 中使用 **clarify**: Blocked tools 列表包含 clarify, 子 agent 无法向用户请求澄清
- 2. 禁止向共享记忆写入: 子 agent 无法调用 memory 工具, 防止竞态条件破坏 MEMORY.md
- 3. 工具集权限不扩大: 子 agent 的工具集是父工具集与配置请求的交集

13.3 性能注意事项

- 1. 子 agent 构建在主线程: _build_child_agent() 在主线程执行, 子 agent 的 get_tool_definitions() 耗时会计入主线程
- 2. 线程池大小: HermesAgentLoop 的工具线程池默认为 128, 适用于 89+ 并发 RL 任务
- 3. 凭证解析开销: _resolve_delegation_credentials() 每次委托都重新解析 Provider, 可能有 I/O 开销

十四、文件索引

hermes-agent/

tools/

├─ delegate_tool.py

├─ interrupt.py

└─ mixture_of_agents_tool.py

← Sub-agent 委托核心逻辑

← 全局中断信号

← MoA 独立工具

agent/

├─ auxiliary_client.py

└─ display.py

← Provider/Model 解析路由

← KawaiiSpinner UI

run_agent.py

← AIAgent 主类 (含中断传播)

environments/

└─ agent_loop.py

← RL 环境专用异步 Agent Loop

acp_adapter/

├─ session.py

└─ entry.py

← ACP 会话管理

← ACP 入口

batch_runner.py

← 批量 AIAgent 并行执行

rl_cli.py

← RL 训练 CLI

📄